



Raport cercetare GMRV semestrul 2

The Rift We Share

Masterand:

Codarcea Alexandru-Christian

Îndrumător:

Boiangiu Costin Anton

Motto:

Two Paths. One Split Screen. Never The Same Run.

1. Generarea Procedurală a Spațiului (GPC)

Baza tehnică a jocului o reprezintă un sistem robust de generare procedurală [2] [3] a dungeon-urilor, gândit pentru a asigura rejucabilitate infinită și coerență spațială [1].

1.1. Partiționarea Binară a Spațiului (BSP) și Sistemul Grid-Based

Arhitectura nivelurilor este construită folosind algoritmul BSP (Binary Space Partitioning). Acesta împarte recursiv un spațiu bidimensional inițial, generând o ierarhie de noduri până la atingerea dimensiunilor minime, formând astfel „frunzele” arborelui, care devin camerele efective ale dungeon-ului. Întregul sistem este fundamentat pe o matrice (grid-based layout), unde fiecare „tile” (pătrat) reprezintă o unitate spațială bine definită. Această structură grid-based va facilita în viitor [4] sisteme complexe de spawn (de exemplu, inamici sau jucători asociați matematic centrului unui tile specific) [Figura 1].

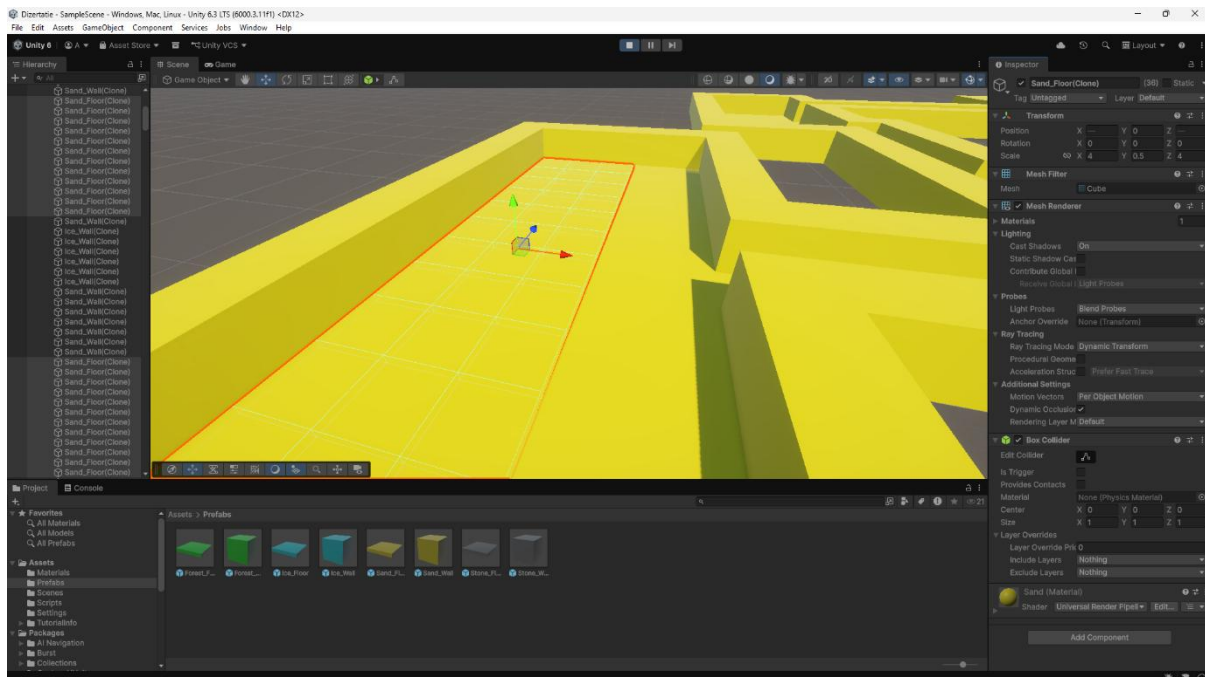


Figura 1: Structura grid-based a nivelului procedural, ilustrând împărțirea modulară a spațiului în „tiles” pentru facilitarea calculelor de pathfinding și a sistemelor de coordonate la spawn.

Pentru a oferi un grad înalt de modularitate și testare, algoritmul a fost expus în interfața motorului grafic printr-o serie de parametri ajustabili [Figura 2]:

- **Grid Width / Grid Depth:** Definesc suprafața totală a matricei în care se va genera nivelul.
- **Min Partition Size:** Stabilește pragul minim la care algoritmul se oprește din a mai tăia/diviza o secțiune de spațiu.
- **Min Room Size:** Determină dimensiunea fizică minimă pe care o poate avea o cameră în interiorul partiției sale.

Ajustarea acestor parametri permite generarea instantanee a unor configurații variate (ex: multe camere mici și aglomerate vs. puține camere vaste).

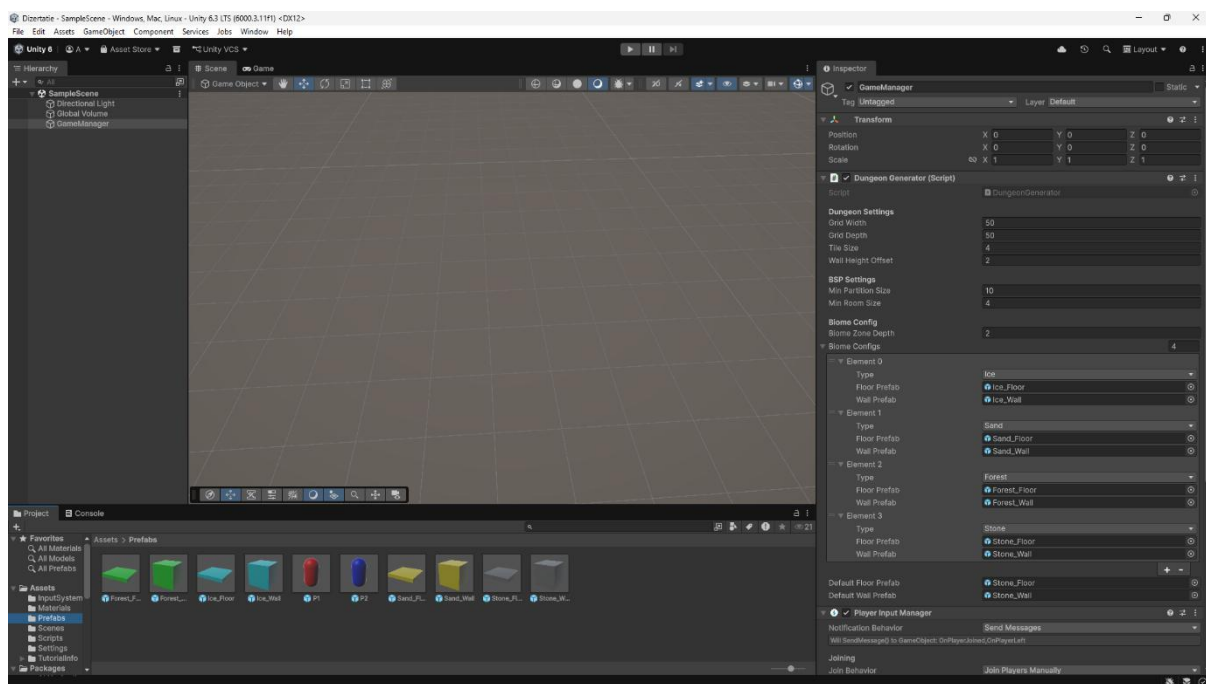


Figura 2: Interfața scriptului de generare procedurală (Dungeon Generator), expunând parametrii ajustabili (Grid Size, Min Room Size, Biome Zone Depth) care permit controlul iterativ asupra arhitecturii și densității labirintului.

1.2. Sistemul de Biome-uri (Moștenire Ierarhică)

Pentru a evita o distribuție haotică [5] a tematicilor vizuale, am dezvoltat un algoritm de **Atribuire prin Moștenire Ierarhică (Hierarchical Property Inheritance)** aplicat pe arborele binar. În loc să asigneze un biom aleatoriu

fiecărei camere individuale, algoritmul selectează o adâncime specifică în arbore (depth) și asignează biome-urile „strămoșilor”. Toate camerele descendente moștenesc automat această proprietate, generând cartiere sau zone extinse cu o estetică unitară (ex: Zone de Gheață, Pădure, Deșert) [Figura 3]. Parametrul **Biome Zone Depth** [Figura 2] controlează la ce adâncime în arborele BSP se decide tematica unei regiuni.

Sistemul este conceput să fie extrem de scalabil; deși în stadiul actual de prototip diferențierea este strict vizuală, arhitectura permite adăugarea facilă de noi biome-uri ce vor influența activ gameplay-ul pe viitor.

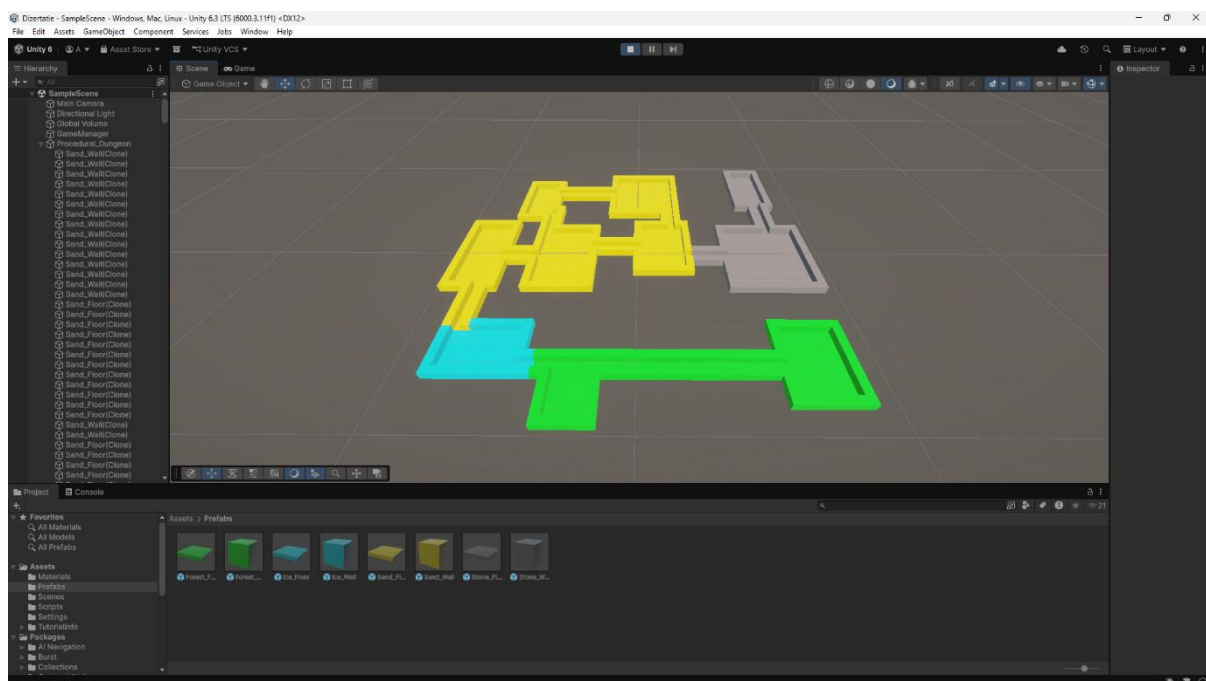


Figura 3: Vedere de ansamblu asupra labirintului generat procedural, evidențiind aplicarea sistemului de biome prin algoritmul de moștenire ierarhică (zonele fiind diferențiate cromatic).

1.3. Conectivitatea (Algoritmul A*) și Blending-ul Coridoarelor

Pentru a garanta un *pathing* valid între toate camerele generate, am implementat algoritmul de pathfinding A* (A-Star) [6] [7]. Acesta calculează cea mai scurtă distanță logică între centrele camerelor. O decizie majoră de design a fost stabilirea regulilor de cost pentru acest algoritm: traversarea unui spațiu complet gol are un cost mare (moveCost = 3f), în timp ce traversarea unei podele deja generate are un cost minim (moveCost = 1f).

Această regulă forțează algoritmul să prefere interconectarea drumurilor și suprapunerea peste coridoarele deja existente. În plus, prin parcurgerea și conectarea camerelor în ordinea generării lor (secvențial), rezultatul este un dungeon extrem de bine structurat și organic.

O inovație tehnică la nivel vizual o reprezintă **blending-ul de coridoare** [Figura 4]. Când A* unește două camere aparținând unor biome-uri diferite, coridorul rezultat își adaptează dinamic textura: prima jumătate preia materialul zonei de start, iar a doua pe cel al destinației, creând o tranziție vizuală clară ("hard-cut" 50/50) direct pe structura de grid.

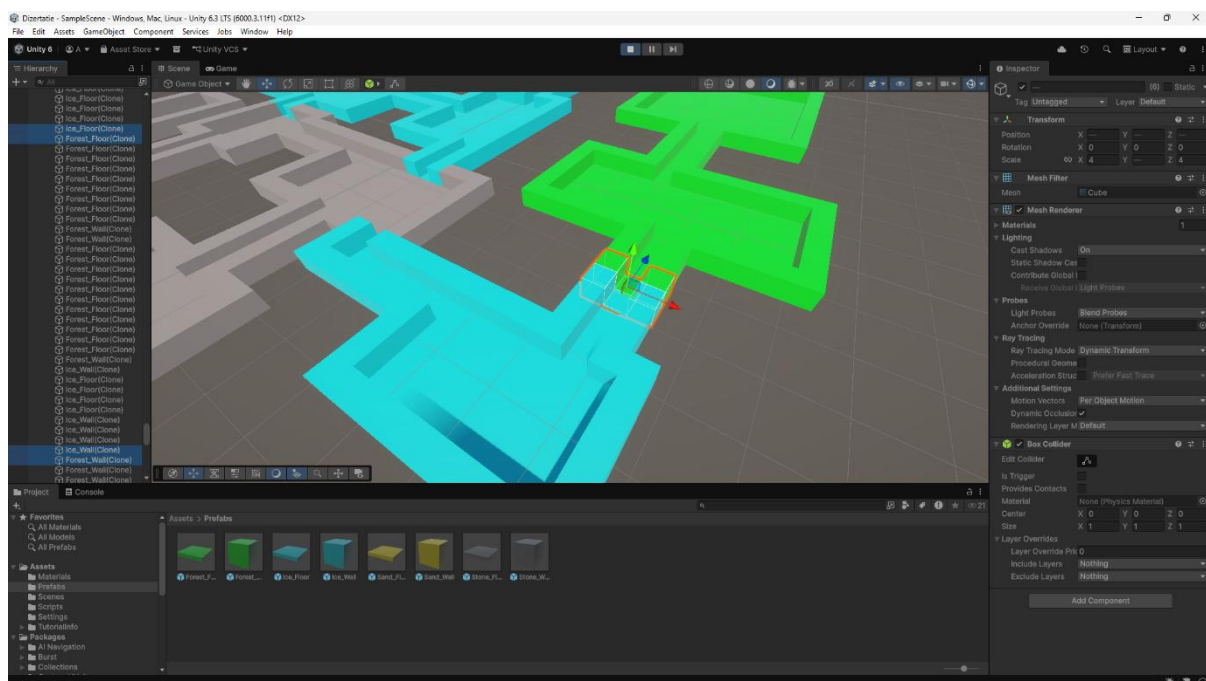


Figura 4: Detaliu al algoritmului A* în acțiune: generarea coridorului de legătură și tranziția vizuală (blending 50/50) pe grid între două zone cu biomi diferiți.

2. Arhitectura de Multiplayer Local (Split-Screen)

Nucleul experienței de cooperare este susținut de o tranziție completă către **Unity New Input System** [8], care abstractizează citirea datelor hardware prin „Control Schemes” și „Actions” (mapping-uri).

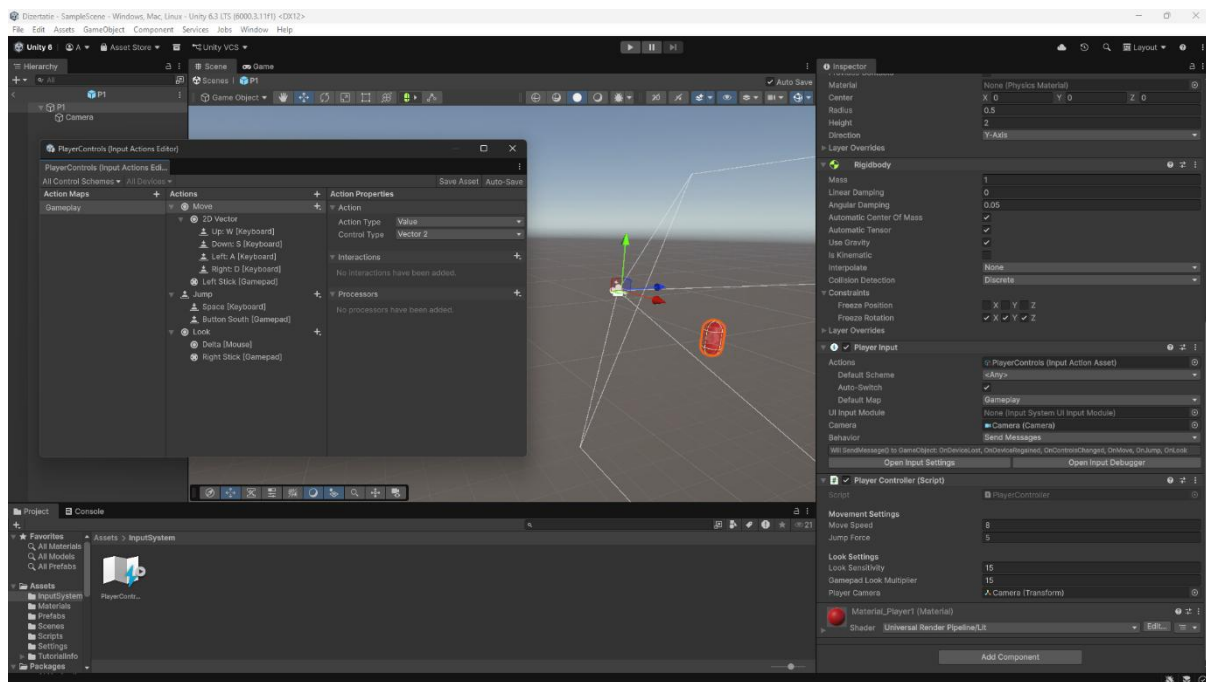


Figura 5: Configurarea noului sistem de Input din Unity, ilustrând maparea acțiunilor și utilizarea schemelor de control distincte (Gamepad / Keyboard) esențiale arhitecturii split-screen.

2.1. Viewport Independent și Instanțierea Jucătorilor

Sistemul asociază fiecărui jucător instanțiat propriul Viewport și propria cameră fizică, controlate prin componenta **PlayerInput** configurată cu scheme specifice (Keyboard / Gamepad) [Figura 5]. Rezultatul este un split-screen vertical redat simultan, permițând celor două capsule (personajele prototip) să exploreze independent dungeon-ul, având libertate totală de deplasare și săritură [Figura 6].

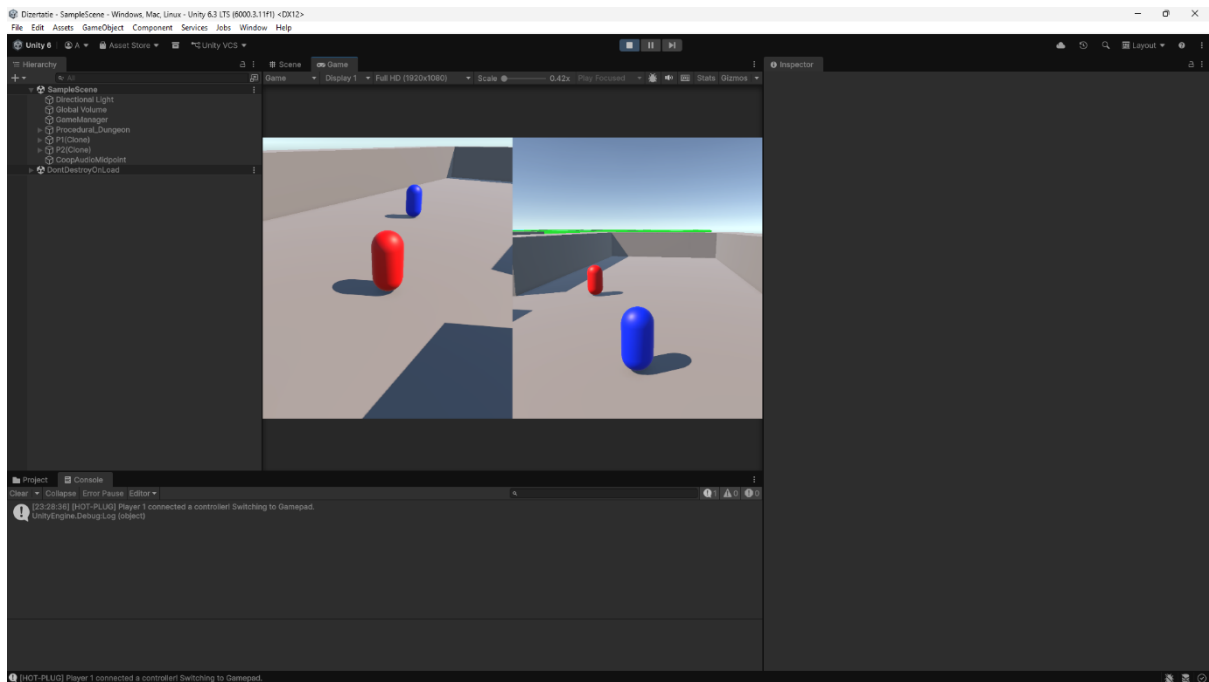


Figura 6: Implementarea funcțională a viewport-ului de tip split-screen vertical, prezentând cei doi jucători (reprezențați ca prototipuri) navigând independent în nivelul generat.

2.2. Gestionarea Avansată a Dispozitivelor (Hot-Plugging & Edge Cases)

Pentru a oferi o experiență de "Couch Co-op" fluidă și stabilă, sistemul a fost supus unor teste amănunțite de *hot-plugging* (conectare/deconectare în timp real) pentru a gestiona multiple cazuri limită (edge cases):

- **Prioritizare Controller:** Jocul favorizează mereu folosirea gamepad-ului. La pornire, dacă există un singur controller conectat, acesta îi este atribuit automat Jucătorului 1 (P1), iar Jucătorul 2 (P2) primește fallback pe tastatură. Dacă ambele controllere sunt conectate inițial, ele sunt asiguate corespunzător ambilor jucători.
- **Dynamic Hot-Plugging:** Sistemul ascultă evenimentele OS-ului. Conectarea unui al doilea controller în timpul sesiunii deblochează un *Smart Switch*: P2 renunță automat la tastatură și preia noul gamepad, fără a întrerupe starea de joc.
- **Safe Fallback & Prevenția Erorilor:** Dacă un controller este deconectat accidental (ex: baterie descărcată), sistemul execută un *Safe Fallback* pe tastatură. Mai mult, a fost implementată o validare

de siguranță (`Keyboard.current != null`) pentru a preveni excepțiile de tip `ArgumentNullException` în cazul în care un periferic (precum mouse-ul) este deconectat fizic pentru a elibera porturi USB. De asemenea, sistemul verifică disponibilitatea tastaturii și comută instantaneu jucătorul afectat pe taste, doar dacă acestea nu sunt deja ocupate de celălalt jucător.

3. Sisteme Suport de Imersiune

3.1. Arhitectura Audio Cooperativă (Midpoint Audio Listener)

O limitare tehnică majoră a motorului Unity este imposibilitatea de a randa corect sursele audio spațiale 3D atunci când există mai mult de un `AudioListener` [9] activ în scenă. Pentru a rezolva acest conflict specific jocurilor split-screen, a fost concepută o arhitectură cu un **Midpoint Audio Listener**. Sistemul generează o entitate invizibilă (`CoopAudioMidpoint`) care își calculează și actualizează poziția constant la jumătatea distanței matematice dintre cei doi jucători [Figura 7]. Astfel, sunetul 3D se propagă corect în boxe/căștile jucătorilor, echilibrând peisajul auditiv indiferent de distanța fizică dintre caractere în dungeon.

Deși această metodă echilibrează auditiv scena în majoritatea timpului, s-a observat că, în situațiile în care cei doi jucători se află la distanțe extrem de mari unul față de celălalt în labirint, percepția direcțională a sunetului devine mai puțin intuitivă.

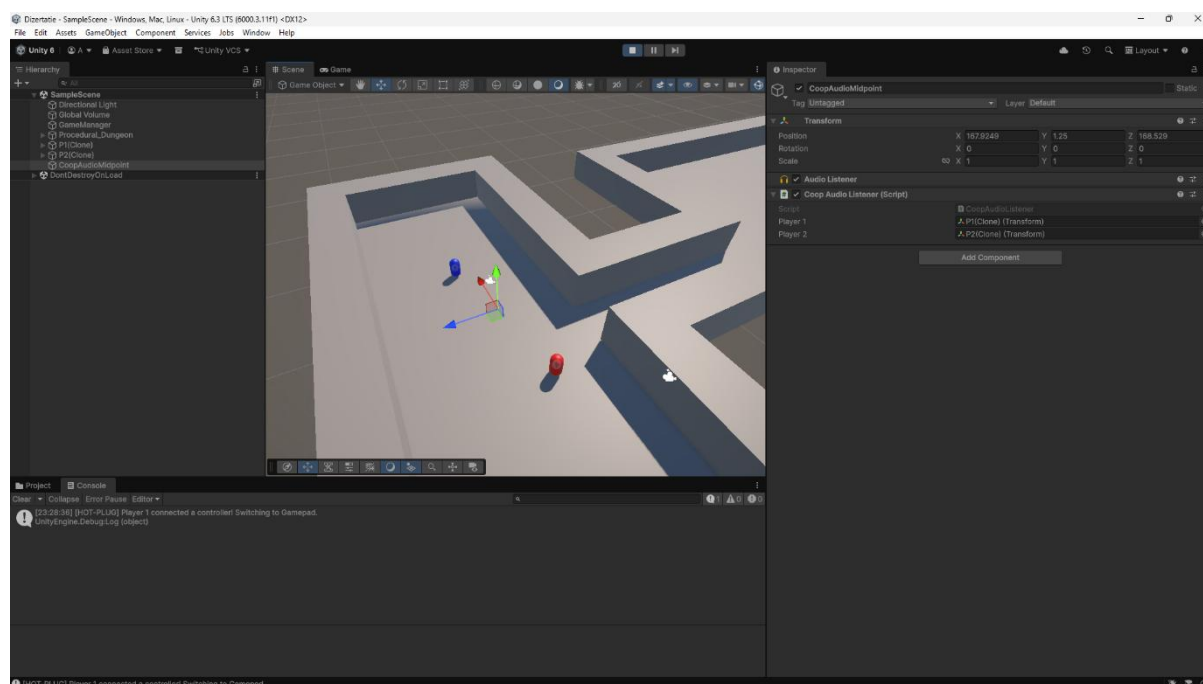


Figura 7: Arhitectura sistemului audio pentru co-op local: instanțierea obiectului invizibil „`CoopAudioMidpoint`”, plasat dinamic la jumătatea distanței dintre jucători pentru rezolvarea limitărilor motorului Unity.

4. Evaluare și Testare

Prototipul a fost supus unui proces riguros de evaluare tehnică și de gameplay:

- **Integritatea Generării:** Au fost generate și testate zeci de hărți (seeds). În toate iterațiile, sistemele A* și BSP au funcționat impecabil; nu au existat bug-uri, camere izolate/neconectate, probleme de "clipping" (trecere prin texturi) sau elemente geometrice lipsă (podele/ziduri).
- **Sistemul de Spawn:** Logica de spawn a demonstrat consistență, plasând de fiecare dată jucătorii corect, într-o cameră validă selectată aleatoriu din dungeon, calculând un offset de siguranță față de centrul camerei.
- **Performanță:** Generarea nivelului și a arborelui se realizează instantaneu în stadiul actual. S-a notat însă că, odată cu viitoarele adăugiri de prop-uri, inamici și interacțiuni complexe, generarea va deveni mai costisitoare („demanding”) din punct de vedere hardware, putând dura câteva secunde.
- **Feedback extern:** Prototipul a fost testat alături de verii mei Pescariu Matei și Pescariu Vlad (care vor oferi asistență QA pe tot parcursul dezvoltării). Feedback-ul acestora a fost extrem de pozitiv, confirmând că obiectivele fundamentale ale prototipului (stabilitatea co-op-ului și explorarea procedurală) au fost atinse cu succes.

5. Pași Următori și Concluzii

Prototipul realizat în semestrul 2 a demonstrat cu succes fezabilitatea nucleului tehnic: fuziunea dintre generarea procedurală a unui mediu 3D interconectat și arhitectura solidă de Input necesară unui joc split-screen modern. Următoarele etape de dezvoltare se vor concentra pe transformarea acestui fundament tehnic într-o experiență completă de joc.

Planuri de viitor (Semestrul 3):

- **Extinderea Zonei de Gameplay (Combat & Progresie):** Implementarea unui sistem de luptă "fast-paced" adaptat pentru split-screen, crearea inteligenței artificiale pentru inamici și dezvoltarea sistemului de meta-progresie pentru jucători.
- **Mecanici de Biome:** Trecerea biome-urilor de la stadiul pur vizual la un stadiu interactiv, unde tematica regiunii va influența direct și activ regulile de gameplay (ex: efecte de alunecare pe gheață, slow-down în deșert).
- **Optimizare Audio:** Documentarea și testarea unor soluții avansate pentru a perfecționa mixajul audio 3D, atenuând limitările identificate la distanțe mari între jucători.

Bibliografie

- [1] Santa Monica Studio, Liz Barlow. *Valhalla Reinvented: Designing a Roguelite in God of War Ragnarök*, URL: <https://sms.playstation.com/stories/valhalla-reinvented-designing-a-roguelite-in-god-of-war-ragnarok> [accesat: 10.06.2026]
- [2] N. Shaker, J. Togelius, M. J. Nelson. *Procedural Content Generation in Games*. Springer, 2016.
- [3] T. X. Short, T. Adams. *Procedural Generation in Game Design*. A K Peters/CRC Press, 2017.
- [4] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson, 2008.
- [5] Ronald T. Kneusel. *Random Numbers and Computers*. Springer, 2018.
- [6] S. J. Russell, P. Norvig. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020.
- [7] I. Millington. *AI for Games (3rd Edition)*. CRC Press, 2019.
- [8] Unity Technologies. *Input System Manual*, URL: <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.19/manual/index.html> [accesat: 11.06.2026]
- [9] Unity Technologies. *Audio Listener Documentation*, URL: <https://docs.unity3d.com/Manual/class-AudioListener.html> [accesat: 11.06.2026]